



Problema A. Página en blanco intencional

Nombre del archivo fuente: Intentional.c, Intentional.cpp, Intentional.java, Intentional.py Estándar

Aporte:

Producción: Estándar

Algunos concursos copian el conjunto de ejercicios por ambas caras (es decir, anverso y reverso). Sin embargo, no comienzan el ejercicio en la contraportada. Si un ejercicio requiere un número impar de páginas (1, 3, 5, 7, etc.), dejan la última página (la contraportada) en blanco para que el siguiente ejercicio comience en la anverso (normalmente se incluye el mensaje «Esta página se dejó en blanco intencionalmente» en la última página del ejercicio).

Dado el número de problemas y el número de páginas para cada problema, determine cuántas páginas quedan en blanco en el conjunto de problemas, es decir, cuántas páginas tienen el mensaje especial.

Entrada

La primera línea de entrada contiene un entero, p ($1 \leq p \leq 20$), que indica la cantidad de problemas en el conjunto.

Cada una de las siguientes p líneas de entrada contiene un número entero (entre 1 y 100, inclusive) que indica el número de páginas de un problema.

Salida

Imprime el número de páginas en blanco.

Ejemplo

Entrada	Salida 3
5 13 10 70 1 1	
3 6 12 8	0
1 1	1



Problema B. Full House

Nombre del archivo fuente: Completo.c, Completo.cpp, Completo.java, Completo.py
Aporte: Estándar
Producción: Estándar

Una mano de póquer consta de 5 cartas. La mano se llama full cuando tres cartas coinciden en valor y las dos cartas restantes también coinciden en valor (pero el segundo valor es diferente del primero).

Dado un número entero que consta de exactamente cinco dígitos, determine si es un full (tres dígitos, un valor, dos dígitos otro valor).

Aporte

Solo hay una línea de entrada; proporciona un número entero entre 10000 y 99999, inclusive.

Producción

Imprima SÍ o NO, indicando si el entero de entrada es un full o no.

Ejemplo

Entrada	Producción
12121	SÍ
12345	NO
88888	NO
77555	SÍ



Problema C. Pelotas antiestrés de pato

Nombre del archivo fuente: Duck.c, Duck.cpp, Duck.java, Duck.py Estándar

Aporte:

Producción: Estándar

El Equipo de Programación de la UCF organiza un concurso presencial para las escuelas secundarias de Florida. Este concurso sigue el modelo del ICPC y cada equipo está formado por tres estudiantes. En el concurso más reciente, los organizadores compraron pelotas antiestrés (patos) para regalar como obsequios. Los patitos son de varios colores, así que decidieron darle a cada equipo tres patitos de diferentes colores; es decir, cada equipo tendrá un patito de diferentes colores. El proceso se volvió un poco engorroso, ¡así que aquí estás para ayudar!

Dados los colores de los patos y la cantidad de cada color, determine si los tres estudiantes de cada equipo pueden conseguir patos de diferentes colores. Suponga que hay $3n$ patos en total para repartir entre n equipos.

Entrada

La primera línea de entrada contiene un número entero, c ($3 \leq c \leq 10$), que indica la cantidad de colores diferentes de patos. Cada una de las siguientes c líneas de entrada contiene un número entero (entre 1 y 60, ambos inclusive) que indica la cantidad de patos de un color determinado. De nuevo, habrá un total de $3n$ patos para repartir entre n equipos.

Salida

Imprimir Sí o NO, indicando si cada equipo puede obtener o no tres patos de diferentes colores.

Ejemplo

Entrada	Producción
5 2 3 3 2 2	Sí
3 59 1 60	NO
4 3 3 3 3 3	Sí

Explicación: En el

segundo ejemplo, hay 120 patos que se dividirán entre 40 equipos. No es posible que cada equipo tenga tres patos de diferentes colores.



Problema D. Números gemelos

Nombre del archivo fuente: Twin.c, Twin.cpp, Twin.java, Twin.py Estándar

Aporte:

Producción: Estándar

Un número entero es gemelo si cada dígito en la posición $2n - 1$ en el número es el mismo que el dígito en la posición $2n$, por ejemplo, 88, 8855, 775588, 8888. Tenga en cuenta que un número gemelo debe tener un número par de dígitos.

Dado un rango (límite inferior y límite superior), determine cuántos gemelos hay en ese rango.

Entrada :

Hay dos líneas de entrada: la primera proporciona el límite inferior del rango y la segunda, el límite superior. Suponga que $10 \leq \text{límite inferior} \leq \text{límite superior} \leq 10100000$.

Salida:

Imprima el número de gemelos en el rango dado. Como el resultado puede ser grande, imprima el valor módulo $104 + 7$.

Ejemplo

Entrada	Salida 1
20 30	
22 99	8
1000 4489	39
330210 1234567890	9683

Explicación El

número de gemelos para el último rango de ejemplo es 19690, por lo que la salida será $19690 \bmod 10007 = 9683$.



Problema E. Casa sencilla

Nombre del archivo fuente: Simple.c, Simple.cpp, Simple.java, Simple.py Estándar

Aporte:

Producción: Estándar

Ha adquirido un terreno rectangular. Los límites del terreno no son necesariamente paralelos a los ejes x e y . Quiere construir una casa rectangular en su terreno, pero desea que los límites de la casa sean paralelos a los ejes x e y . La casa puede tocar los límites del terreno, pero no sobrepasarlos.

También te gustaría que la casa se vea bien, por lo que debe tener una relación fija entre su ancho (eje x) y su altura (eje y).

Dadas las coordenadas de las cuatro esquinas del terreno, así como dos números enteros w y h (que representan la relación entre el ancho de la casa y su altura), determina el área de la casa más grande que puedes construir.

Aporte

Hay cinco líneas de entrada:

- Cada una de las primeras cuatro líneas de entrada contiene dos enteros, x e y $-10 \leq x, y \leq 10$, lo que proporciona una esquina del terreno. Las cuatro esquinas se muestran en sentido horario, comenzando por la esquina inferior izquierda.
- La quinta línea de entrada contiene dos números enteros, w y h ($1 \leq w, h \leq 20$), que representan la relación entre Ancho a alto.

Salida:

Imprime el área de la casa más grande que puedas construir. Se aceptará cualquier respuesta con un error absoluto o relativo de 10^{-6} respecto a la respuesta correcta.

Ejemplo

Entrada	Salida
3 0 0 4 8 10 11 6 2 1	12.5
0 0 0 5 100 5 100 0 20 1	500.0



Problema F. Pathfinder y los lobos

Nombre del archivo fuente: Pathfinder.c, Pathfinder.cpp, Pathfinder.java, Pathfinder.py Estándar

Aporte:

Producción: Estándar

El famoso explorador, Moose The Pathfinder (MTP), se encuentra en la celda (1, 1) de un laberinto rectangular de $r \times c$ y necesita llegar a la celda (r, c). MTP puede ir en cualquiera de las cuatro direcciones (norte, sur, este, oeste), excepto cuando se encuentra en una celda limítrofe (estas celdas limítrofes tienen menos opciones de movimiento, ya que MTP no puede cruzar las paredes exteriores del laberinto). Desafortunadamente, hay lobos escondidos en algunas celdas y, obviamente, MTP quiere mantenerse lo más alejado posible de ellas.

Dado el laberinto, determine la distancia más cercana que MTP tendrá que recorrer a cualquier celda con lobos al intentar ir de la posición (1, 1) a la posición (r, c). Nótese que MTP recorrería una distancia mayor si le resultara más fácil estar más lejos de las celdas con lobos; es decir, MTP tomaría una ruta más larga si eso implicara estar más lejos de las celdas con lobos. La distancia de MTP a una celda con lobo se define como el número de pasos (en las cuatro direcciones) desde donde se encuentra MTP hasta la celda con lobo.

Supongamos que habrá un camino para MTP que no pasará por ninguna celda con lobo, es decir, lo más cerca que MTP tendrá que llegar a una celda con lobo será a una distancia de 1. Supongamos también que hay al menos una celda con lobo en cada laberinto.

Entrada:

La primera línea de entrada contiene dos enteros: r $2 \leq r \leq 103$, que indica el número de filas del laberinto, y c $2 \leq c \leq 103$, que indica el número de columnas. El laberinto se proporciona en las siguientes r líneas de entrada. Cada una de estas líneas de entrada contiene c caracteres a partir de la columna 1. Las celdas con un lobo se indican con la letra W y las celdas vacías con un guion (-). Suponga que las posiciones inicial y final estarán vacías.

Salida

Imprima el MTP más cercano que tendrá que llegar a cualquier celda con lobo.

Ejemplo

Entrada	Salida 2
5 5 —O —W—W —O — W—	
3 5 —WW — —	2
3 3 — —O —	1
3 3 —W— — —	1



Explicación En el

tercer ejemplo, MTP está a una distancia de 1 de una celda con lobo cuando está en el destino.

En el último ejemplo, MTP está a una distancia de 1 de una celda con lobo cuando está en la posición inicial.



Problema G. Festival de las Flores

Nombre del archivo fuente: Festival.c, Festival.cpp, Festival.java, Festival.py Estándar

Aporte:

Producción: Estándar

En el Reino de Floralia, el Festival Anual de las Flores marca la llegada de la primavera con una preciada tradición: cada hogar debe exhibir una guirnalda de exactamente N flores en su puerta. Cada año, el rey anuncia K colores de flores diferentes disponibles en los jardines reales, y los ciudadanos pueden usar cualquier combinación de estos colores para crear su decoración (una guirnalda de flores).

El festival de este año tiene un significado especial. El oráculo real ha emergido del Templo de los Pétalos con una visión profética: una subcadena sagrada S de colores que promete traer prosperidad y buena fortuna al reino. Según la profecía, cada cadena de flores debe contener este patrón bendito (subcadena contigua) en algún punto de su disposición, lo que significa que los colores de S deben aparecer consecutivamente y en el mismo orden en algún punto de las flores N . Solo entonces la bendición divina llegará a cada hogar de Floralia.

El asistente del rey, un hombre meticuloso que se enorgullece de la organización del festival, se enfrenta a su reto anual. Cree firmemente que cada hogar merece una decoración única (una guirnalda de flores), y que no hay dos familias que reciban arreglos idénticos en este día tan especial. En sus minuciosos registros, dos guirlandas de flores se consideran iguales si cada una de las N flores tiene el mismo color al mirarse de izquierda a derecha.

Con los valores de N y K de este año anunciados, y la subcadena profética S del oráculo revelada al amanecer, el asistente se dirige a usted con una pregunta urgente: ¿exactamente cuántas cadenas distintas de N flores que contengan S son posibles?

La respuesta determinará si el reino cuenta con suficientes diseños únicos para que todos sus hogares participen en esta bendita tradición. El tiempo apremia y el festival comienza al atardecer.

Dados N , K y S , determine el número de cadenas de flores únicas que contienen la subcadena contigua sagrada.

Entrada

Solo hay una línea de entrada; contiene dos números enteros N $1 \leq N \leq 104$ y K ($1 \leq K \leq 26$), seguidos de la subcadena S ($1 \leq |S| \leq \min(N, 1000)$).

Supongamos que los colores de las flores están representados por las primeras K letras minúsculas, por ejemplo, si $K = 4$, los colores de las flores serán a , b , c y d . La subcadena S también está formada por las primeras K letras minúsculas.

Salida

Imprima un entero, el número de cadenas únicas de N flores que contienen la subcadena sagrada S . Dado que la respuesta puede ser bastante grande, su programa debe informar el número como el resto no negativo más pequeño cuando se divide por $109 + 7$.

Ejemplo

Entrada	Salida 15
4 2 a	
2 3 ca	1
10 4 abc	127248
5 26 ¡Guau!	2027
10 26 ¡Guau!	176416529
31 2 a	147483633



Problema H. Fiesta de cumpleaños

Nombre del archivo fuente: Cumpleaños.c, Cumpleaños.cpp, Cumpleaños.java, Cumpleaños.py Estándar

Aporte: Estándar

Producción: Estándar

A Bob le encantan los cumpleaños. De hecho, le encantan tanto que ha llevado sus celebraciones a otros planetas.

Cada vez que aterriza en un nuevo planeta, su primera misión (antes de la comida, el agua o el aire respirable) es organizar la fiesta de cumpleaños más grande que la galaxia haya visto jamás. ¿El truco? Bob solo celebra cuando se representan todos los cumpleaños.

En este planeta, el año consta de exactamente n días, y cada persona que conoce Bob tiene la misma probabilidad de nacer en cualquiera de esos días. Partiendo de una lista de invitados vacía, Bob añade a su grupo uno por uno, seleccionando a cada invitado de forma aleatoria y uniforme. Se detiene en cuanto su lista incluye a alguien nacido en cada uno de los n cumpleaños posibles.

¿Cuál es el número esperado de personas que Bob necesita invitar hasta tener un conjunto completo de cumpleaños?

Dado un entero n , encuentre el número esperado de personas seleccionadas aleatoriamente necesarias hasta que cada uno de los n cumpleaños posibles esté representado al menos una vez. El cumpleaños de cada persona se distribuye de forma uniforme e independiente a lo largo de los n días. Calcule esta respuesta mod $109 + 7$.

Entrada

Solo hay una línea de entrada; proporciona un número entero n $1 \leq n \leq 2 \cdot 10^9$ este, el número de días que tiene un año planeta.

Salida

Imprima el número esperado de personas mod $109 + 7$. Si la respuesta es una fracción, imprímala como un entero en la forma $A \cdot B^{-1} \text{ mod } (109 + 7)$, donde B^{-1} es el inverso modular de $B \text{ mod } (109 + 7)$.

Ejemplo

Entrada	Salida 1
1	
2	3
6	900000021
10	289682571
14	980613883
20	26325398
1075	742969056
99998	924541619

Explicación

Para el ejemplo de entrada $n = 6$, tendremos:

$A = 147$

$B = 10$

Respuesta = $A \cdot B^{-1} \text{ mod } (109 + 7) = 900000021$

Para el ejemplo de entrada $n = 20$, tendremos:

$A = 279175675$

$B = 3879876$

Respuesta = $A \cdot B^{-1} \text{ mod } (109 + 7) = 26325398$



Problema I. Cosas oscuras

Nombre del archivo fuente: Dark.c, Dark.cpp, Dark.java, Dark.py Estándar

Aporte:

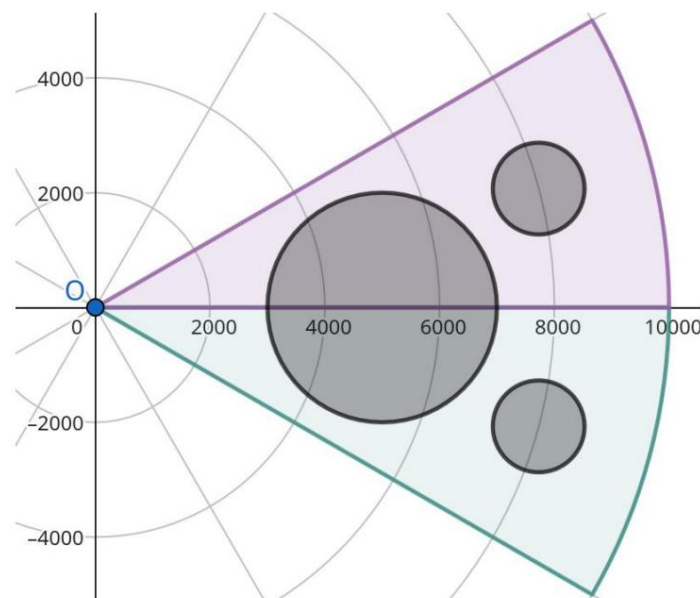
Producción: Estándar

Mientras estás sentado en el observatorio contemplando las estrellas, suenan varias alertas que llaman tu atención hacia la consola principal, que dice:

Advertencia: ¡Se han avistado varios objetos no identificados!

Estos objetos son bastante peculiares, pues no se corresponden con ninguna propiedad física de ninguna sustancia o energía conocida o teorizada, ¡ni siquiera con la materia oscura ni la energía oscura! Por supuesto, lo primero que piensas es en cuántos artículos excelentes podrías publicar y, al igual que muchos grandes físicos, empiezas por darles a estos objetos un nombre perfectamente apropiado: "¡Cosas Oscuras!".

Ahora que has nombrado estos objetos, has decidido centrarte en los detalles menos importantes, como cómo interactúan con las masas y la energía circundantes. Para estudiarlos con más detalle, decides emplear k sensores en los sistemas de telescopios del observatorio y quieres que cada uno de estos sensores realice una cantidad de trabajo equivalente. Para determinar cómo asignar el espacio de búsqueda de cada sensor, has decidido modelar tu situación mediante la geometría, como se ilustra a continuación:



En el plano cartesiano, se modela la Tierra como un punto en el plano ubicado en el origen. En el momento actual, se sabe que cualquier punto que se encuentre a R pársecs de la Tierra y a un máximo de θ grados del eje x positivo puede ser visto por el observatorio. Los objetos oscuros también pueden modelarse como círculos no intersecantes cuyos centros están a r_i pársecs de la Tierra, a α_i grados del eje x positivo y tienen un diámetro de d_i . Debido a la limitación de la tecnología del observatorio, también se sabe que cada objeto oscuro debe estar completamente contenido dentro del conjunto de puntos observables.

Cada sensor escaneará todos los puntos que no estén contenidos en ningún objeto oscuro, que se encuentren a R parsecs de la Tierra y que se encuentren dentro de un rango de ángulos medidos desde el eje x positivo. Para asegurar que todos los sensores realicen el mismo trabajo, se deben hallar $k - 1$ ángulos ψ_j tales que el primer sensor escanee de $-\theta$ a ψ_1 , el segundo de ψ_1 a ψ_2 , el tercero de ψ_2 a ψ_3 , y así sucesivamente hasta el último escáner, que escanee de ψ_{k-1} a θ y las áreas del conjunto de puntos que escanea cada sensor sean iguales.



Dada la región de puntos observables y la ubicación de cada cosa oscura, divida el espacio de búsqueda en k áreas iguales, donde se ignora cualquier punto dentro de una cosa oscura.

Entrada

La primera línea de entrada contiene cuatro números enteros: n ($1 \leq n \leq 103$), que indica la cantidad de elementos oscuros, k ($2 \leq k \leq 50$), que indica la cantidad de sensores, R ($104 \leq R \leq 106$), que indica la distancia máxima de cualquier punto observable, y θ ($30 \leq \theta \leq 60$), que indica el ángulo máximo de cualquier punto observable cuando se mide desde el eje x positivo.

Cada una de las siguientes n líneas de entrada contendrá tres enteros: r_i ($1 \leq r_i \leq R$), i ($|i| \leq \theta$) y d_i ($1 \leq d_i \leq 2000$), que indican la ubicación y el tamaño del i -objeto oscuro. Se garantiza que dos objetos oscuros no se toquen ni se intersequen, y que cada uno de ellos esté completamente contenido dentro del conjunto de puntos observables.

Salida:

Imprime $k-1$ números de punto flotante, uno por línea de salida, indicando el ángulo de cada partición. Se aceptará cualquier resultado con un error absoluto o relativo de 10^{-6} .

Ejemplo

Aporte	Producción
3 2 10000 30 5000 0 2000 8000 15 800 8000 -15 800	0.0000000000
3 3 10000 60 9000 55 50 9500 -59 10 9250 57 30	-20.0009600000 19.9979900000

Explicación: La

imagen de la página anterior corresponde al primer ejemplo, donde dividimos el conjunto de puntos observables en dos regiones iguales (las secciones verde y morada). En este escenario, ambos sensores cubrirán un área de aproximadamente $5,0286 \times 10^7$ pársecs².



Problema J. Viaje Galáctico

Nombre del archivo fuente: Galactic.c, Galactic.cpp, Galactic.java, Galactic.py Estándar

Aporte:

Producción: Estándar

Eres el navegante a bordo de la nave Celestial Wanderer, con destino al agujero negro Sagitario A en el , el supermasivo centro de la galaxia. Tu nave está equipada con un motor de curvatura cuántica de última generación.

A diferencia de los sistemas de propulsión convencionales que proporcionan empuje continuo, este motor de curvatura funciona mediante traslaciones espaciales discretas. El motor ha sido calibrado con S configuraciones de curvatura distintas, cada una de las cuales puede transportar su nave exactamente seis unidades astronómicas hacia adelante en su trayectoria.

Este motor solo funciona mientras viajas hacia el centro de la galaxia, por lo que tendrás que equipar un motor diferente para el viaje de regreso.

En la sala de observación, escuchas a un grupo de exploradores veteranos hablar sobre una antigua red de agujeros de gusano que atraviesa la galaxia siguiendo trayectorias radiales hacia su centro. Según estos exploradores, existen P series distintas de agujeros de gusano a lo largo de la trayectoria recta desde nuestra ubicación actual hasta el centro de la galaxia. Cada serie P forma un patrón regular de entradas de agujeros de gusano, y los agujeros de la serie i se encuentran a distancias de $0, \pi i, 2\pi i, 3\pi i, \dots$ unidades astronómicas de tu posición actual.

Da la casualidad de que estamos en un lugar donde convergen todas las series P .

El grupo de exploradores veteranos procedió a detallar con precisión cómo funcionan estos agujeros de gusano. Se puede entrar en cualquier agujero de gusano de cualquier serie y salir de cualquier agujero de gusano de cualquier serie (incluso de la misma), con dos restricciones cruciales. En primer lugar, solo se puede avanzar hacia el centro galáctico, lo que significa que si se entra en un agujero de gusano a una distancia X , se debe salir a una distancia mayor que X . En segundo lugar, las distorsiones gravitacionales limitan cada salto de agujero de gusano a un máximo de K unidades astronómicas, lo que significa que la distancia entre los puntos de entrada y salida no puede superar K .

El control de misión planea colocar un puesto de control de reabastecimiento a lo largo de esta transitada ruta hacia Sagitario A . Para garantizar que la estación sea accesible para todas las naves que utilicen el mismo motor de curvatura que la suya, quieren que calcule la distancia máxima D que no puede alcanzar desde su punto de partida.

Dadas las configuraciones de curvatura de la nave S , el conjunto de series de agujeros de gusano P y el valor K en unidades astronómicas, determine la distancia máxima D desde su ubicación actual que no puede alcanzar mediante alguna combinación de motor de curvatura y travesías de agujeros de gusano.

Aporte

Hay tres líneas de entrada:

- La primera línea de entrada contiene tres números enteros: S, P y K ($1 \leq S, P, K \leq 10$), que indican (respectivamente) el número de configuraciones de deformación, el número de intervalos de agujero de gusano y el valor de K .

segunda línea de entrada contiene S enteros, $s_1, s_2, \dots, s_S$, donde $1 \leq s_i \leq 104$ • La , el conjunto de motor warp distancias.

- La tercera línea de entrada contiene P enteros, $p_1, p_2, \dots, p_P$ $1 \leq p_i \leq 104$, el conjunto de intervalos de agujero de gusano.

Salida:

Imprime la distancia D , en unidades astronómicas, que representa la distancia máxima inalcanzable. Si todas las ubicaciones son alcanzables o hay infinitas ubicaciones inalcanzables, imprime -1 .



Ejemplo

Entrada	Salida 5
2 2 4 3 7 4 11	
3 2 1 6 9 15 4 10	-1
3 2 5 8 17 20 10 15	63

Explicación Explicación

del primer ejemplo Entrada/Salida: $S = \{3, 7\}$: Esto significa que desde su

posición inicial 0, puede avanzar a 3, 6, 9, . . . ; también puede avanzar desde su posición inicial 0 a 7, 14, 21, . . . Además, puede combinar estos pasos hacia adelante en cualquier orden, p. ej., desde la posición inicial 0 puede recorrer 3 pasos seguidos de 7 pasos para llegar a la posición 10.

$P = \{4, 11\}$: Esto significa que hay agujeros de gusano en las posiciones 0, 4, 8, etc., así como en las posiciones 0, 11, 22, etc. Con $K = 4$, se puede atravesar del portal 0 a la posición 4, luego de la 4 a la 8 y, finalmente, de la 8 a la 11; se puede ir de la 8 a la 11 porque la distancia entre el punto de entrada (8) y el punto de salida (11) no supera K (4). Cabe destacar que no se puede atravesar del portal 0 a la posición 11 directamente (aunque P contenga 11) porque la distancia entre los puntos de entrada y salida supera K .

El valor más grande que es completamente inalcanzable a través de alguna combinación de configuraciones de deformación en S o agujeros de gusano en P es 5 (esto significa que podemos alcanzar 6, 7, 8, 9, 10, . . .).

Notas:

- Cuando estés en una posición (sin importar cómo hayas llegado allí), puedes usar los pasos en S para avanzar. Para Por ejemplo, después de entrar en un agujero de gusano, puedes usar los pasos en S para avanzar.
- Los valores en P no se pueden utilizar como pasos para avanzar, es decir, los valores de agujero de gusano no se pueden utilizar como pasos. Por ejemplo, en el primer ejemplo anterior, supongamos que usted se mueve a la posición 7; desde allí no puede usar 11 (un valor en P) para avanzar a la posición 18 (por supuesto, puede haber otras formas de llegar a la posición 18).
- Puedes avanzar desde una posición de agujero de gusano a cualquier otra posición de agujero de gusano (suponiendo que el con-se cumple la restricción K).



Problema K. Pasarelas deterioradas

Nombre del archivo fuente: Degenerando.c, Degenerando.cpp, Degenerando.java, Degenerando.py Estándar

Aporte:

Producción: Estándar

Exploras una antigua estación espacial abandonada buscando la manera de volver a casa tras una avería en tu nave. La estación espacial consta de n nodos y $n - 1$ pasarelas bidireccionales. Cada pasarela bidireccional conecta dos nodos diferentes, y cualquier par de nodos está conectado directa o indirectamente entre sí.

Tras un buen rato explorando la estación, los generadores de energía se activan, revelando una Astro-Reliquia Tecnológica Sustentable (STAR) en cada centro. Hay exactamente una STAR (un número entero) en cada centro. Estas STAR son justo lo que buscabas, ya que puedes usarlas para estabilizar la tecnología de portal que trajiste de tu nave espacial y te ayudarán a volver a casa. Más específicamente, adjuntarás estas STAR a tu portal siguiendo el patrón (orden) de una secuencia objetivo. La tarea se simplifica al no tener valores duplicados en la secuencia objetivo; es decir, sabes que $x_i = x_j$ en la secuencia objetivo para cualquier índice i y j tal que $i = j$.

Las STAR vienen en una de k formas (k valores enteros diferentes) y, por su formación en ingeniería, usted sabe que ciertos conjuntos de STAR se pueden conectar a la tecnología del portal, lo que da como resultado diversas estabilidades. si el valor i coincide con el i La STAR adjunta al portal tiene la forma x_i (es decir, el valor entero x_i). Más específicamente, siguiente elemento de la secuencia objetivo, la estabilidad del portal aumenta en uno. Tenga en cuenta que la secuencia objetivo determina qué se puede adjuntar al portal. Si el elemento i de la secuencia objetivo adjunto al portal debe ser x_i , la adopción de x_i , la y_o cualquier otra STAR provocará la desintegración completa del portal.

Tu objetivo ahora es recolectar estrellas para estabilizar la tecnología de tu portal y viajar a casa. Como no sabes bien dónde te encuentras (en qué centro) en la estación espacial, sí conoces su distribución, así que para recolectar un conjunto de estrellas, tendrás que recorrerla aleatoriamente.

Desafortunadamente, la reactivación de la energía ha dificultado el desplazamiento por la estación espacial. Debido a la antigüedad y la falta de mantenimiento de las pasarelas, cualquier pasarela se deteriorará y no podrá volver a utilizarse; es decir, cada pasarela solo se puede usar una vez. Por lo tanto, recolectarás STAR con la siguiente estrategia:

1. En el centro actual, toma la ESTRELLA en el centro si es de la forma que necesitas, es decir, coincide con la siguiente elemento en la secuencia de destino.
2. Si has acumulado t estrellas en total hasta ahora, intenta usar el portal. El portal funcionará con una probabilidad de éxito de $1 - \frac{t+1}{pq}$, donde p y q son constantes dadas.
3. Si el paso 2 no funciona, dirígete a un nuevo centro. De las pasarelas que funcionan, elige una equiprobable. Si no hay pasarelas conectadas al centro en el que te encuentras, permanecerás en él y repetirás el paso 2 (es decir, intentarás usar el portal de nuevo) hasta que lo consigas, es decir, hasta que el portal te lleve a casa.

Ahora quieres determinar la estabilidad esperada del portal al llevarlo a casa. Como no estás seguro de en qué centro te encuentras, asumirás que cada centro tiene la misma probabilidad de ser el centro de partida.

Teniendo en cuenta la disposición de la estación espacial, la secuencia objetivo de STAR a recolectar y el tipo de STAR que contiene cada centro, determine la estabilidad esperada de un portal que se pueda crear.



Entrada

La primera línea de entrada contiene cinco números enteros: n $1 \leq n \leq 105$, que indica el número de concentradores en la estación espacial, m $1 \leq m \leq 105$, que indica el tamaño de la secuencia objetivo de STAR, k $1 \leq k \leq 109$, que indica el número de tipos de STAR, y p y q ($0 < p < q < 998, 244, 353$), que indican las constantes dadas utilizadas para calcular la probabilidad de éxito.

La segunda línea de entrada proporciona la secuencia objetivo; es decir, esta línea de entrada constará de m enteros donde la i -ésimo entero corresponde a la i -ésima ESTRELLA objetivo. Cada uno de estos enteros estará entre 1 y k , ambos inclusive. y ningún par de números enteros será igual.

La siguiente línea de entrada estará compuesta por n números enteros, donde i -ésimo Un entero indica el tipo de STAR en el concentrador. Cada tipo de STAR se denotará con un entero entre 1 y k , ambos inclusive. Tenga en cuenta que diferentes

Los concentradores pueden tener el mismo tipo STAR.

Cada una de las siguientes $n - 1$ líneas de entrada contendrá dos números enteros u y v ($1 \leq u, v \leq n$) que denotan que hay una pasarela que conecta el centro u y el centro v . Tenga en cuenta que la estación espacial está completamente conectada, es decir, puede llegar a cualquier centro desde cualquier otro centro.

Salida:

Imprima la estabilidad esperada del portal después de usarlo correctamente. Dado que la respuesta se puede representar, denota como $\frac{a}{b}$ donde a y b son números enteros, salida $a(b^{-1} \bmod 998, 244, 353)$, donde b^{-1} el multiplicativo inverso de $b \bmod 998, 244, 353$.

Ejemplo

Entrada	Salida
2 2 2 1 2 2 1 1 2 1 2	124780545
4 5 6 4 5 3 5 2 1 6 5 2 3 2 1 2 1 3 1 4	343422678

Explicación La

respuesta del primer ejemplo corresponde a la fracción $7/8$ y la respuesta del segundo ejemplo corresponde a la fracción $26947/37500$.



Problema L. Copogonia

Nombre del archivo fuente: Copogonia.c, Copogonia.cpp, Copogonia.java, Copogonia.py Estándar

Aporte:

Producción: Estándar

Copogonia es un país con un conjunto de n ciudades, que actualmente cuenta con n carreteras. El conjunto de carreteras forma un polígono convexo, de ahí el nombre del país. Desafortunadamente, algunos ciudadanos se han quejado de los tiempos de viaje entre ciudades. Debido a la limitada red vial, siempre hay dos maneras de viajar entre cualquier par de ciudades distintas (en sentido horario o antihorario), y para algunos pares de ciudades, incluso la ruta más corta parece demasiado larga.

Después de muchas rondas de estudios cuidadosos, se ha determinado que los ciudadanos estarían contentos si ninguna distancia de viaje individual entre un par de ciudades excediera la longitud m , es decir, el camino entre dos ciudades tuviera una longitud m o menos.

El gobierno de Copogonia ha realizado una investigación exhaustiva y ha decidido que podría construir hasta k nuevas carreteras, cada una conectando directamente dos ciudades que antes no estaban conectadas mediante una carretera directa, con una distancia igual a la distancia en línea recta entre ellas. Sin embargo, cada carretera conlleva un costo de desarrollo. Naturalmente, el gobierno de Copogonia desea minimizar el costo del desarrollo y, al mismo tiempo, mantener la satisfacción de todos sus ciudadanos. Le han encomendado la tarea de encontrar el subconjunto preciso de carreteras que se deben desarrollar para lograr sus objetivos.

Dadas las ubicaciones en el plano cartesiano de las n ciudades de Copogonia (en el orden en que están conectadas con caminos, notando que también hay un camino desde la última ciudad a la primera), una lista de k caminos adicionales potenciales que podrían agregarse (par de ciudades y costo para construir el camino), y la distancia máxima, m , que los ciudadanos están dispuestos a viajar en un solo viaje entre pares de ciudades, determine el costo mínimo de desarrollar un subconjunto de caminos que mantendrá felices a los ciudadanos de Copogonia.

Entrada

La primera línea de entrada contiene tres números enteros separados por espacios: n ($4 \leq n \leq 50$), que indica el número de ciudades en Copogonia, k ($1 \leq k \leq 10$), que indica el número de caminos adicionales potenciales para construir, y m , que indica la distancia máxima de viaje entre cualquier par de ciudades que los copogonianos están dispuestos a tolerar.

una de las siguientes n líneas de entrada contiene dos enteros separados por espacios, x_i e y_i ($0 \leq x_i \leq 10^4$, $0 \leq y_i \leq 10^4$), que indican la ubicación de la ciudad i en la cuadrícula cartesiana (suponiendo que no hay dos ciudades en la misma ubicación). Hay una carretera recta que conecta la ciudad i con la ciudad $i + 1$ ($1 \leq i \leq n - 1$) y una carretera que conecta las ciudades n y 1 . Las carreteras pueden recorrerse en cualquier dirección.

Las posibles carreteras que se pueden añadir se indican en las siguientes k líneas de entrada. Cada una de estas k líneas de entrada contiene tres enteros separados por espacios: u , v ($1 \leq u, v \leq n$, $u \neq v$) y c ($1 \leq c \leq 10^4$), lo que indica que se podría añadir una carretera entre la ciudad u y la ciudad v con un coste de c . Se garantiza que actualmente no existe ninguna carretera de la ciudad u a la v y que ninguna de las posibles carreteras enumeradas se encontrará entre el mismo par de ciudades.

Supongamos que:

- La entrada es tal que se debe agregar al menos una carretera, es decir, sin carreteras adicionales los ciudadanos están no contento (al menos uno de los caminos más cortos es mayor que la longitud m).
- Si se construyen todos los caminos potenciales, los ciudadanos estarán contentos, es decir, todos los caminos más cortos tendrán una longitud m o menos.
- El valor de m será tal que sumar o restar 0,001 a m no modificará la respuesta.



Salida

Imprima el costo mínimo de construir un conjunto de carreteras que mantendrían felices a todos los ciudadanos de Copogonia.

Ejemplo

Entrada	Salida
4 2 15 0 0 0 10 10 10 10 0 1 3 1000 2 4 500	1500
5 3 2003 0 0 0 100 1000 100 2000 99 1000 0 2 4 50 4 1 123 3 5 47	170



Problema M. Jardinería de fin de semana

Nombre del archivo fuente: Weekend.c, Weekend.cpp, Weekend.java, Weekend.py Estándar

Aporte:

Producción: Estándar

Tu compañero de piso está planeando algunos proyectos para el fin de semana para mejorar tu jardín. El plan es comprar plantas. Quiere gastar al menos L dólares y como máximo H dólares. Hay tres tipos de plantas: baratas, normales y caras. Desafortunadamente, no puedes distinguirlas con solo mirar una planta. Sabes que cada tipo tiene un precio diferente y cuántas de cada tipo tiene tu vivero local.

Ahora estás pasando tu tiempo caminando por los pasillos del vivero, escogiendo una planta, llevándola al cajero para que la cobre y rezando para permanecer dentro del costo total deseado (tu presupuesto asignado).

Si no está dentro del costo total deseado (es decir, si su costo total es demasiado bajo o demasiado alto), su compañero de habitación se frustrará.

Teniendo en cuenta el costo y la cantidad de cada tipo de planta junto con su rango de presupuesto deseado, determine la probabilidad de que pueda comprar plantas una a una y permanecer dentro de su rango de costo total.

Entrada

La primera línea de entrada contiene dos números enteros: L y H ($1 \leq L \leq H \leq 109$), que representan el extremo inferior y el extremo superior del presupuesto, respectivamente.

La segunda línea de entrada contiene tres números enteros: C , N y E ($1 \leq C < N < E \leq 109$), que representan el costo de las plantas baratas, normal y cara, respectivamente.

La tercera línea de entrada contiene tres números enteros: c , n y e ($1 \leq c, n, e \leq 100$), que representan el número de plantas baratas, normales y caras, respectivamente.

Salida:

Imprima la probabilidad de completar la compra con éxito dentro del presupuesto deseado, eligiendo las plantas una a una. Su respuesta será aceptada si se encuentra dentro de 10^{-6} de la respuesta correcta.

Ejemplo

Entrada	Salida
30 60 5 15 70 3 1 1	0.2
40 60 5 15 70 3 1 1	0.0